

Lab Introduction

Qihang Xu (徐启航) / Cao'er Wang (王曹尔)

2025/09/10

Choice: JOS or Tacos^{New}

- **JOS Overview**

- JOS schedule & grading
- Some tips
- Hands-on Lab 1: Bootloader

- Alternative: Tacos
- Tacos schedule & grading
- Tools & Tips
- A peek to Lab 0: Appetizer
- Comparisons

JOS Overview

News

- **Sep 1:** Please sign up for [Piazza 6.828](#) to discuss labs, lectures and papers. We will look at Piazza regularly and answer questions (unless one of you answers first); the entire class can see and benefit from these exchanges.

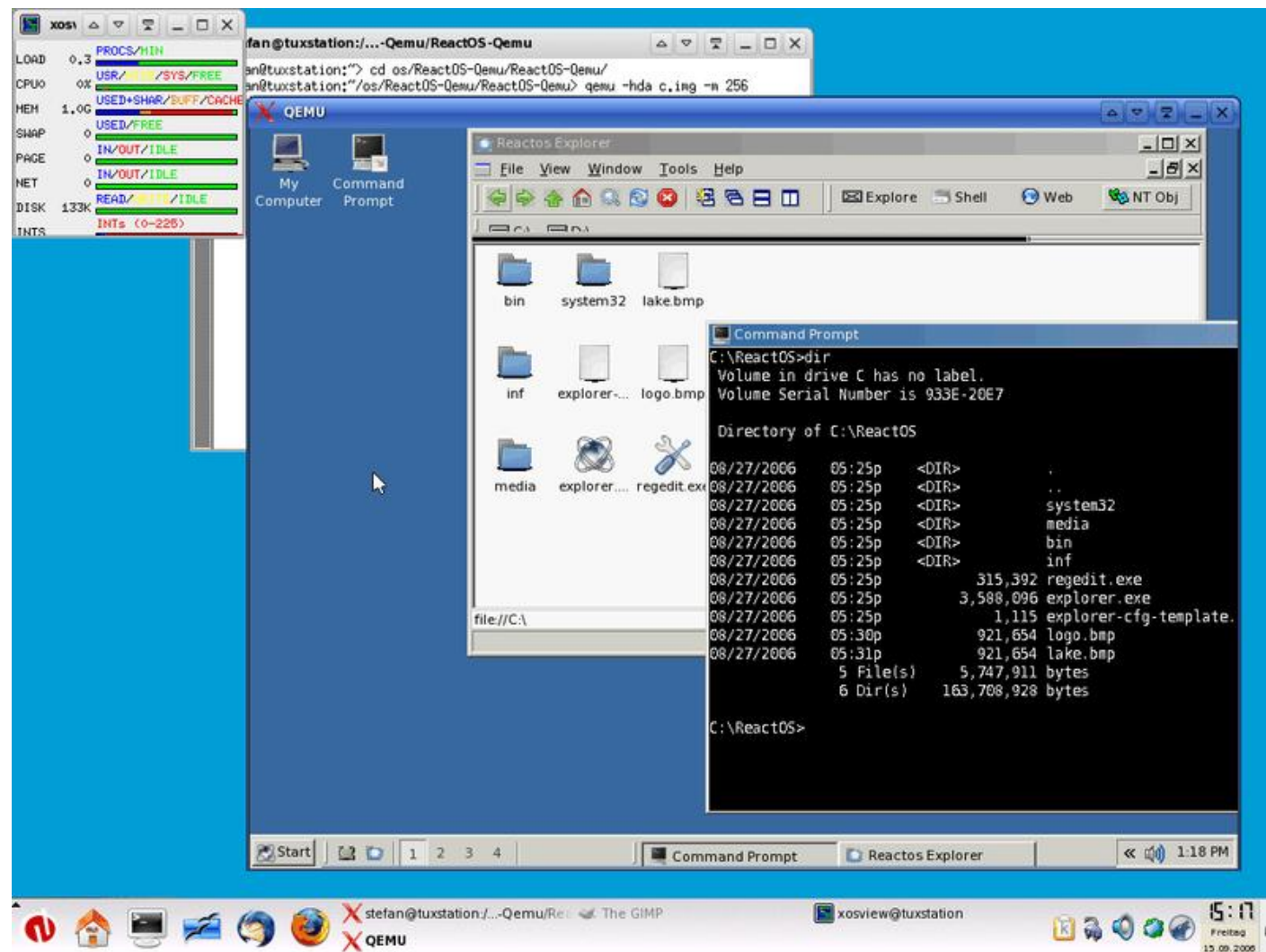
Questions or comments regarding 6.828? Send e-mail to the TAs at 6828-staff@lists.csail.mit.edu.

 [Creative Commons License](#) [Top](#) // [6.828 home](#) // Last updated Friday, 03-Jul-2020 10:56:27 EDT

- JOS is an x86-based OS designed by MIT for teaching
 - See <https://pdos.csail.mit.edu/6.828/2018/> ([back-up at course.pku.edu.cn](#))
 - Note that we use course materials of **2018**, not 2019 or 2020
- To finish JOS, several labs are provided, in which we'll implement important OS components from scratch
 - Lab 1: bootloader
 - Lab 2: virtual memory management
 - Lab 3: user mode and system call
 - Lab 4: multitasking
 - Lab 5: file system
- We'll understand OS concepts better if we implement them

Lab environment

- JOS is a real OS that can run on real x86 hardware
- JOS will take over all hardware during runtime
- We cannot debug JOS as a common application
- Instead, we use **QEMU**, an emulator provides hardware emulation



- See <https://pdos.csail.mit.edu/6.828/2018/tools.html> and <https://pdos.csail.mit.edu/6.828/2018/labguide.html> for more details

Outline

- JOS Overview
 - **JOS schedule & grading**
 - Some tips
 - Hands-on Lab 1: Bootloader
-
- Alternative: Tacos
 - Tacos schedule & grading
 - Tools & Tips
 - A peek to Lab 0: Appetizer
 - Comparisons

Lab Schedule

Lab	Start time	Report Submission	Weeks
Lab1	2025.9.10	2025.9.28	3 weeks
Lab2	2025.9.29	2025.10.19	3 weeks
Lab3	2025.10.20	2025.11.2	2 weeks
Lab4	2025.11.3	2025.11.23	3 weeks
Lab5	2025.11.24	2024.12.14	3 weeks

Quiz (Open Book)

- Objective: check the completion and the understanding of labs
- Schedule
 - There are one quiz in this semester
 - Quiz will be scheduled after lab5: 2025.12.22

OS Lab: Grading (30%)

- Complete the lab requirements: 60%
 - Complete the coding tasks
 - Submit the report
 - Due date is hard deadline!
 - Deduct 10% for each late day (3 days maximum for each lab)
- Quiz : 40%
- Bonuses: 10%
 - One or more challenge for each lab
 - Choose from a given list of challenges

Outline

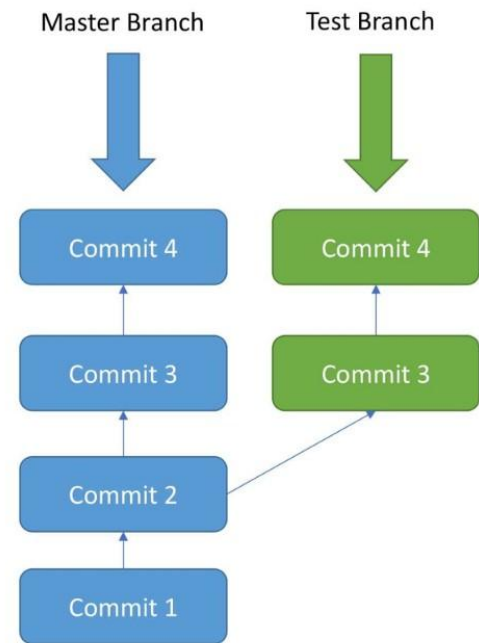
- JOS Overview
 - JOS schedule & grading
 - **Some tips**
 - Hands-on Lab 1: Bootloader
-
- Alternative: Tacos
 - Tacos schedule & grading
 - Tools & Tips
 - A peek to Lab 0: Appetizer
 - Comparisons

Some tips

- Start early!
 - Debugging JOS can be time consuming
- **Read documents and understand existing code carefully before writing your own code**
 - JOS official lab guides
 - Intel Software Developer's Manual: <https://software.intel.com/en-us/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4>
 - OSDev Wiki: <https://wiki.osdev.org/>
- Maybe look for bugs in previous labs will help if you encounter strange bugs in current lab
 - JOS has an automated grader, but it only ensures basic correctness

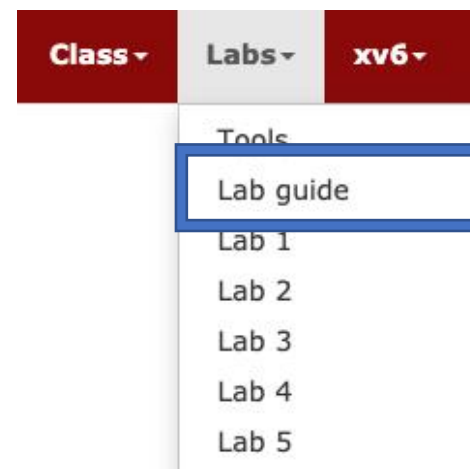
Tools: git

- JOS is a large project, you need to manage your complex code
 - You may forget a previous small change
 - Backup with remote repository
- Frequently used commands
 - Git clone
 - Git add, git commit -m
 - Git status, git log
 - Git reset
 - Git branch / git checkout
 - Git push



Tools

- Makefile
 - Tells ``make'' how to compile and link a program
 - Make qemu
 - Make qemu-nox
 - Make qemu-gdb
 - Make qemu-nox-gdb



```
1:Makefile
helloworld : main.o print.o
    clang -o helloworld main.o print.o
main.o : main.c print.h
    clang -c main.c
print.o : print.c print.h
    clang -c print.c
clean :
    rm helloworld main.o print.o
~
```

- Custom the Makefile to simplify your commands

```
1:print.c
#include "print.h"
void printhello(){
    printf("Hello, world\n");
}
~
1:main.c
#include "print.h"

int main(void){
    printhello();
    return 0;
}
~
1:print.h
#include <stdio.h>

void printhello();
```

Tools

- GDB
- Basic usage:
 - run – r
 - continue – c
 - quit - q
 - next – n
 - step - s
 - breakpoints – b
- plugin makes GDB easy to use
 - GEF, Pwndbg, peda

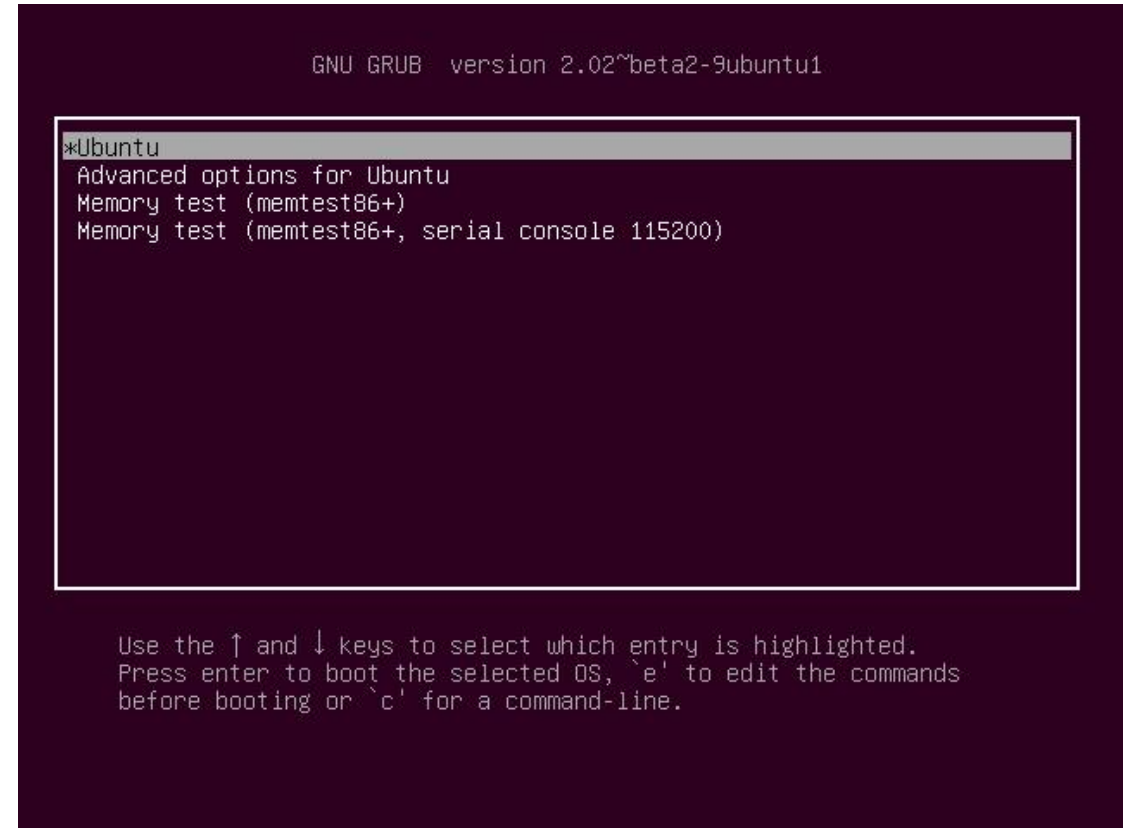
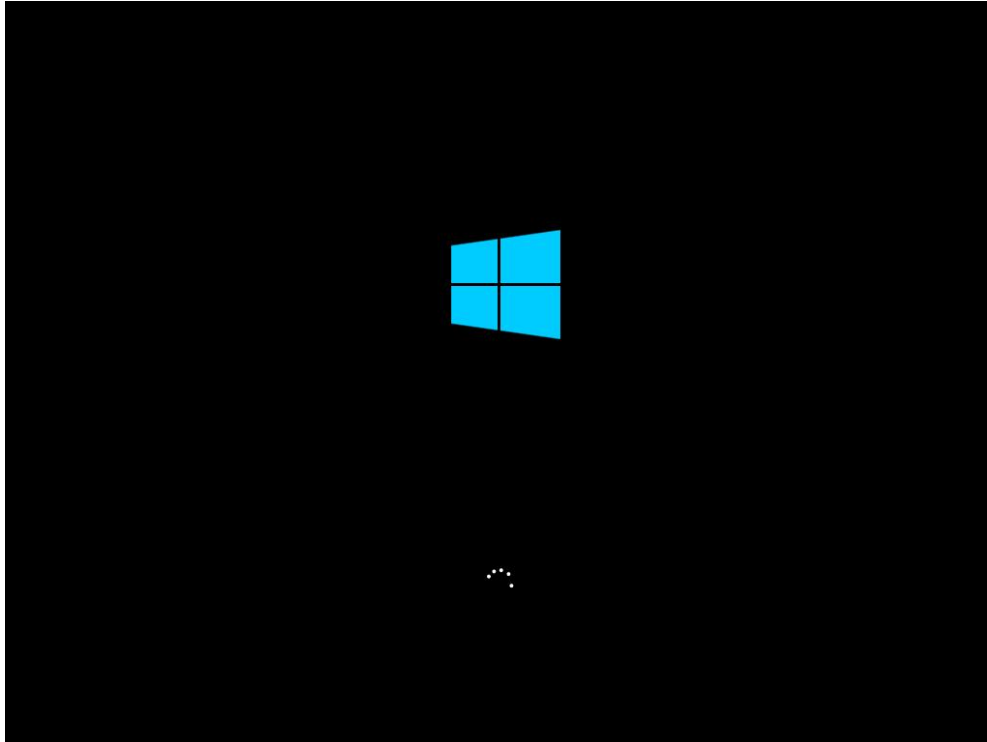
- info br
- info re
- print –
- list – l
- backtr
- x

```
[ Legend: Modified register | Code | Heap | Stack | String ]
registers
$rax : 0x0
$rbx : 0x0
$rcx : 0x00007ffff7ffcca0 → 0x0004095d00000000
$rdx : 0x0
$rsp : 0x00007fffffe530 → 0x0000000000000000
$rbp : 0x00007fffffe560 → 0x00000000004007f0 → <__libc_csu_init+0> push r15
$rsi : 0x00007ffff7dd1b78 → 0x0000000000602000 → 0x0000000000000000
$rdi : 0x20000
$rip : 0x0000000000400799 → <main+64> mov QWORD PTR [rbp-0x28], rax
$r8 : 0x00007ffff7fec700 → 0x00007ffff7fec700 → [loop detected]
$r9 : 0x1
$r10 : 0x0
$r11 : 0x246
$r12 : 0x0000000000400580 → <_start+0> xor ebp, ebp
$r13 : 0x00007fffffe640 → 0x0000000000000001
$r14 : 0x0
$r15 : 0x0
$eflags: [carry PARITY adjust ZERO sign trap INTERRUPT direction overflow resume virtualx86 identification]
$ss: 0x002b $cs: 0x0033 $ds: 0x0000 $gs: 0x0000 $es: 0x0000 $fs: 0x0000
stack
0x00007fffffe530 +0x0000: 0x0000000000000000 ← $rsp
0x00007fffffe538 +0x0008: 0x0000000000000000
0x00007fffffe540 +0x0010: "myfile.txt"
0x00007fffffe548 +0x0018: 0x0000000000007478 ("xt?")
0x00007fffffe550 +0x0020: 0x00007fffffe640 → 0x0000000000000001
0x00007fffffe558 +0x0028: 0xd7c3f14d3cddb000
0x00007fffffe560 +0x0030: 0x00000000004007f0 → <__libc_csu_init+0> push r15 ← $rbp
0x00007fffffe568 +0x0038: 0x00007ffff7a2d830 → <__libc_start_main+240> mov edi, eax
code:i386:x86-64
0x40078c <main+51> mov esi, 0x400874
0x400791 <main+56> mov rdi, rax
0x400794 <main+59> call 0x400550 <fopen@plt>
→ 0x400799 <main+64> mov QWORD PTR [rbp-0x28], rax
0x40079d <main+68> cmp QWORD PTR [rbp-0x28], 0x0
0x4007a2 <main+73> jne 0x4007bc <main+99>
0x4007a4 <main+75> lea rax, [rbp-0x20]
0x4007a8 <main+79> mov rsi, rax
0x4007ab <main+82> mov edi, 0x400876
source:vsprintf.c+20
15 int main ()
16 {
17     FILE * pFile;
18     char szFileName[]="myfile.txt";
19
20     // pFile=0x00007fffffe538 → 0x0000000000000000, szFileName=0x00007fffffe540 → "myfile.txt"
→ 20     pFile = fopen (szFileName,"r");
21     if (pFile == NULL)
22         PrintError ("Error opening '%s'",szFileName);
23     else
24     {
25         // file successfully open
threads
[#0] Id 1, Name: "vsprintf", stopped, reason: SINGLE STEP
trace
[#0] 0x400799 → Name: main()
gef>
```

Outline

- JOS Overview
 - JOS schedule & grading
 - Some tips
 - **Hands-on Lab 1: Bootloader**
-
- Alternative: Tacos
 - Tacos schedule & grading
 - Tools & Tips
 - A peek to Lab 0: Appetizer
 - Comparisons

How does a PC boot?



BIOS

- Recall what we've learned from ICS course
 - Once CPU is powered on, it fetches instructions from memory and modifies data in memory
 - The first instruction is stored in BIOS ROM
 - BIOS ROM is mapped to low-address space of RAM
 - The IBM PC starts executing at physical address `0xffff0`
 - how is the address calculated? `CS:0xf000, PC:0xffff0,  $\$CS \ll 4 + \$PC$`
- CPU then executes pre-defined instructions in BIOS ROM and does some self-checking
- But what's next?

How your PC goes on strike...

```
Intel UNDI, PXE-2.1 (build 083)
Copyright (C) 1997-2000 Intel Corporation

This Product is covered by one or more of the following patents:
US6,570,884, US6,115,776 and US6,327,625

Realtek PCIe GBE Family Controller Series v2.61 (01/07/15)
PXE-E61: Media test failure, check cable

PXE-M0F: Exiting PXE ROM.
No bootable device -- insert boot disk and press any key
```

-

Bootloader

- We need a **bootable device**
- Of course, not all devices (disks) are bootable
- Disks have sectors
 - Hard disks have 512-byte sectors
 - The first sector is used to flag whether the disk is bootable
 - MBR (master boot record) is the first sector ends with **0x55** and **0xAA**. This is manually set.
 - The disk is bootable $\Leftrightarrow$ MBR exists
 - Bootloader are stored in MBR
- BIOS will recognize MBR if it exists on some disk, then load bootloader into RAM and finally jump to the first instruction of it
- `cat <MBR image> | head -c 512 | hexdump [ndisasm -b 16]`
- MBR+BIOS is outdated, but it is easy to understand for tutorial

OS loading

- In bootloader we need to load OS into RAM
- OS are stored in disk. How to load it?
 - x86 has special I/O instructions
 - Consult JOS lab 1 code for more details
- Finally, OS takes over the control flow
- Initializing and we enter the world of OS.

环境上的一些坑

- For Linux & Mac OS: <https://zhuanlan.zhihu.com/p/383308283>
 - 如果编译QEMU时遇到一些问题，可以尝试使用低版本的gcc
- 编译QEMU时提示undefined reference to `major`
 - https://github.com/Ebiroll/qemu_esp32/issues/12
- Lab1中make grade提示QEMU异常退出(terminating on signal 15)
 - 使用Tools提供的QEMU版本
- .stab段是空的
 - 降低GCC版本至9.4

```

Welcome to the QEMU monitor.
Type 'help' for a list of commands.
lab1_Challenge1: Colorful cprintf.
qemu-system-i386: terminating on signal 15 from pid 319214 (make)
ISSING '6828 decimal is 15254 octal!'

ktrace count: OK
ktrace arguments: OK
ktrace symbols: FAIL
AssertionError: got:
```

环境上的一些坑

- Lab3中kern_pgdir变量被异常覆盖
- <https://qiita.com/kagurazakakotori/items/334ab87a6eeb76711936>

Note

For GCC 7 or later, after switching to `lab3` branch an error like `kernel panic at kern/pmap.c:147: PADDR called with invalid kva 00000000` will occur. This is a bug caused by the linker script, modify `kern/kernel.ld` as follow will fix it.

```
--- a/kern/kernel.ld
+++ b/kern/kernel.ld
@@ -50,6 +50,7 @@ SECTIONS
     .bss : {
         PROVIDE(edata = .);
         *(.bss)
+
+         *(COMMON)
         PROVIDE(end = .);
         BYTE(0)
     }
```

环境上的一些坑

- QEMU调试函数传参解析错误
- 在实践中QEMU 9.2版本可以复现该问题，升高版本（15+）或使用指导书上的7.3版本可以解决

```
The target architecture is assumed to be i8086
[f000:fff0] 0xffff0: ljmp $0xf000,$0xe05b
0x0000fff0 in ?? ()
+ symbol-file obj/kern/kernel
(gdb) b cprintf
Breakpoint 1 at 0xf0100a9d: file kern/printf.c, line 27.
(gdb) c
Continuing.
The target architecture is assumed to be i386
=> 0xf0100a9d <cstdio>: endbr32

Breakpoint 1, cprintf (fmt=0xf0101b57 "6828 decimal is %o octal!\n") at kern/printf.c:27
27 {
(gdb) p {$esp, fmt, &fmt}
$1 = {0xf010ffdc, 0xf0101b57, 0xf010ffe0}
(gdb) p fmt
$2 = 0xf0101b57 "6828 decimal is %o octal!\n"
(gdb) si
=> 0xf0100aa1 <cstdio+4>: push %ebp
0xf0100aa1 27 {
(gdb) si
=> 0xf0100aa2 <cstdio+5>: mov %esp,%ebp
0xf0100aa2 in cprintf (
    fmt=0xf01000f0 <i386_init+70> "j\004j\003j\001\215\203j\b\377\377P\350\233\t")
    at kern/printf.c:27
27 {
(gdb) p {$esp, fmt, &fmt}
$3 = {0xf010ffd8, 0xf01000f0 <i386_init+70>, 0xf010ffdc}
(gdb) █
```

Outline

- JOS Overview
- JOS schedule & grading
- Some tips
- Hands-on Lab 1: Bootloader
- **Alternative: Tacos**
- Tacos schedule & grading
- Tools & Tips
- A peek to Lab 0: Appetizer
- Comparisons

Alternative: Tacos

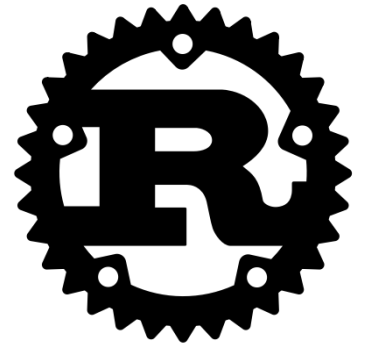
- Tacos is another educational OS (from PKU)
 - <https://pku-tacos.pages.dev/>
- It also have several labs
 - Lab0: Appetizer
 - Lab1: Scheduling
 - Lab2: User Programs
 - Lab3: Virtual Memory
- It is based on **RISC-V**, written in **Rust**
- We'll understand OS concepts better if we implement them



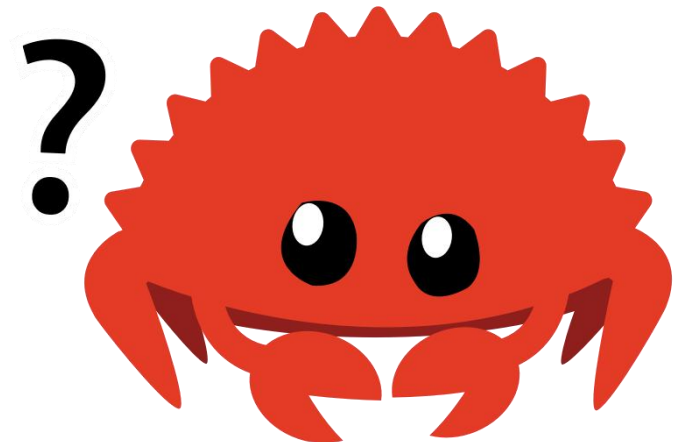
Alternative: Tacos

- Besides labs, you need to do some **extra work**, such as:
 - Port existing applications (redis, gcc, etc.) to Tacos;
 - Implement advanced features for Tacos (eBPF, kernel tracing, etc.);
 - ...
- It depends on your own choice.
- The standard is roughly equivalent to OS competition finals, so the extra work is **quite challenging**.

Wait... What is Rust?



- Rust: A modern system programming language
 - “empowering everyone to build reliable and efficient software”
- Known for its
 - Performance: No GC, memory efficient
 - Reliability: Trait-based type system, Ownership & Lifetimes
 - Productivity: Modern tools & rich documentation
- Adopted in Linux Kernel (Rust for Linux)
- <https://www.rust-lang.org/learn>



Wait... What is RISC-V?

- A open-source & free RISC ISA from Berkeley.
- A Modern ISA: No “legacies”
 - x86: BIOS, UEFI, real & protected & 64-bit modes...
- An upcoming standard for
 - Education & Research
 - Industry adoption



Outline

- JOS Overview
- JOS schedule & grading
- Some tips
- Hands-on Lab 1: Bootloader

- Alternative: Tacos
- **Tacos schedule & grading**
- Tools & Tips
- A peek to Lab 0: Appetizer
- Comparisons

Lab Schedule

Lab	Start time	Report Submission	Weeks
Lab0	2025.9.10	2025.9.21	2 weeks
Lab1	2025.9.22	2025.10.12	3 weeks
Lab2	2025.10.13	2025.11.2	3 weeks
Lab3	2025.11.3	2025.11.23	3 weeks
Extra	2025.11.24	2024.12.22	5 weeks

End-term: Presentation

- Checkpoints:
 - Comprehension of the project
 - Introduction to your extra works
- Form:
 - Simple presentation + face-to-face Q&A
- Grading (30% in all):
 - 60%: 4 labs completion
 - 40%: Extra work + End-term presentation

Outline

- JOS Overview
- JOS schedule & grading
- Some tips
- Hands-on Lab 1: Bootloader

- Alternative: Tacos
- Tacos schedule & grading
- **Tools & Tips**
- A peek to Lab 0: Appetizer
- Comparisons

Tools & Tips

- Docker: A runner for containers <https://www.docker.com/>
 - container: virtual environment + pre-configured application collection
 - commands: pull, build, run, ps, kill, ...
- Rust toolchain - Rustup <https://rustup.rs/>
 - commands: component, toolchain, target, ...
- Rust package manager - Cargo <https://doc.rust-lang.org/cargo>
 - commands: build, run, test, update, ...
- Prioritize pre-built docker images, so that you won't need to configure the environment manually!

Outline

- JOS Overview
 - JOS schedule & grading
 - Some tips
 - Hands-on Lab 1: Bootloader
-
- Alternative: Tacos
 - Tacos schedule & grading
 - Tools & Tips
 - **A peek to Lab 0: Appetizer**
 - Comparisons

A Peek to Lab 0: Appetizer

- Configure & Launch the skeleton project:
 - Install Docker & rust-analyzer extensions in VSCode;
 - `git clone https://github.com/PKU-OS/Tacos.git`
 - Open it in VSCode (“Dev Containers: Open Folder in Container...”)
 - Inside the container: `make` & `cargo run`
- And you will see sth like -->
- And you can write code afterwards.

OpenSBI v1.4-15-g9c8b18e

[illegible]

```
Platform Name      : riscv-virtio,qemu
Platform Features  : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswi
```

```
... (sbi outputs)
```

```
[41 ms] Hello, World!
```

```
[105 ms] Goodbye, World!
```

Outline

- JOS Overview
 - JOS schedule & grading
 - Some tips
 - Hands-on Lab 1: Bootloader
-
- Alternative: Tacos
 - Tacos schedule & grading
 - Tools & Tips
 - A peek to Lab 0: Appetizer
 - **Comparisons**

JOS v.s. Tacos

Comparison	JOS	Tacos
#Labs	5	4 + Extra
Programming Language	C	Rust
ISA	x86	RISC-V
End-term assessment	Quiz	Presentation w/ Q&A
Overall Difficulty	Easier	Harder
Modernity	<i>Oldge</i>	MODERN

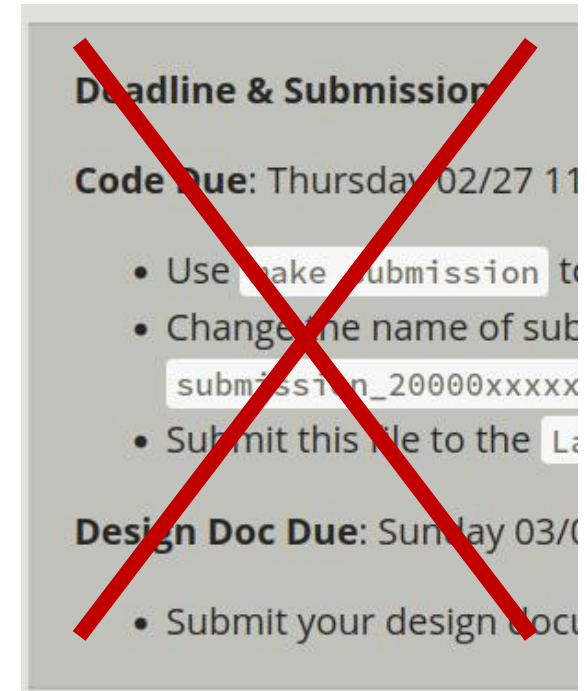
Choose at your own risk!

The first lab has begun

- JOS Lab 1 due: 9/28
- Website: <https://pdos.csail.mit.edu/6.828/2018/labs/lab1/> (**back-up at course.pku.edu.cn**)
- What you need to do
 - Finish lab 1 according to instructions
 - Write lab 1 report
- What to submit
 - Lab 1 source code
 - Lab 1 report (in **PDF** format)
- Where to submit
 - <http://course.pku.edu.cn/>
- **Don't Panic and Start early**

- Tacos Lab 0 due: 9/21
- Website: <https://pku-tacos.pages.dev/lab0-appetizer/tasks>
- What you need to do
 - Finish lab 0 code + design doc according to instructions
 - Write lab 0 report
- What to submit
 - Same as left

DO NOT FOLLOW THIS! That's not our schedule -->



Submission

- Archive both source code and report into **one zip file**
 - ID_name_choice_lab#.zip, for example **2112345678_张三_jos_lab1.zip**
- Report requirement for JOS
 - Recommended that the report should **NOT** exceed 3 pages of A4 paper.
 - For Tacos labs, **include the design doc in the front of the report (counted)**.
 - Describe any particularly noteworthy events you encountered during the lab
 - Architecture design of your code
 - Particularly sophisticated implementation
 - Memorable bugs
 - **State which challenge do you choose at the beginning of the report.**
 - The report format is **PDF**. You can write with Latex, MS Word or Markdown but remember to **transfer the final format into PDF**.

Academic Integrity

- Plagiarizing is strictly prohibited
- YOU CAN NOT:
 - Share your code with others
 - Copy and paste directly from some blogs
 - Ask direct solutions from ChatGPT (你是一个学习操作系统的本科生，正在完成JOS相关实验，请完成lab1)
- YOU CAN:
 - Discuss with your classmates
 - Ask for help from the Internet / ChatGPT / TAs